ECE532: Digital Systems Design
Final Project Group Report

Motion Detection and Security Alert System

by

Jiale Li

Noah Marshall

Kelvin Weng

Liza Zhang

Table of Contents

1. Overview
   - 1.1. Background and Motivation……....1
   - 1.2. Goals……1
   - 1.3. Block Diagram……2
   - 1.4. Brief Description of IPs……2
   - 1.5. System Changes from Proposal……..….3
   - 1.6. Project Complexity……4
2. Outcome
   - 2.1. Results……4
   - 2.2. Future Work and Improvements……..5
3. Project Schedule
   - 3.1. Milestone Comparison Table……5
   - 3.2. Schedule Analysis……7
4. Description of the Blocks
   - 4.1. Ultrasonic Rangefinder Subsystem……8
   - 4.2. OV7670 Camera Subsystem……9
   - 4.3. Camera Data Processing……10
   - 4.4. Block Design and System Integration……10
   - 4.5. VGA Display Subsystem……10
   - 4.6. Audio Alarm Subsystem……11
   - 4.7. Frame Buffer……11
   - 4.8. Overlay ROMs……12
5. Description of Design Tree……12
6. Tips and Tricks……12
7. Video……12
8. References……i

1. Overview

1.1. Background and Motivation

Autonomous sensing and detection systems are widely used in robotics, smart transportation, and safety applications. Obstacle detection plays a critical role in identifying potential dangers and triggering immediate responses. Many existing implementations rely on Arduino-based microcontrollers, which provide simple and quick prototyping but are limited by sequential software execution and offer limited real-time determinism. FPGA-based approaches, such as existing motion detection systems, demonstrate the ability of hardware to process camera data in real time, but are often limited to a single sensing modality and require significant memory for successive video frame comparison.

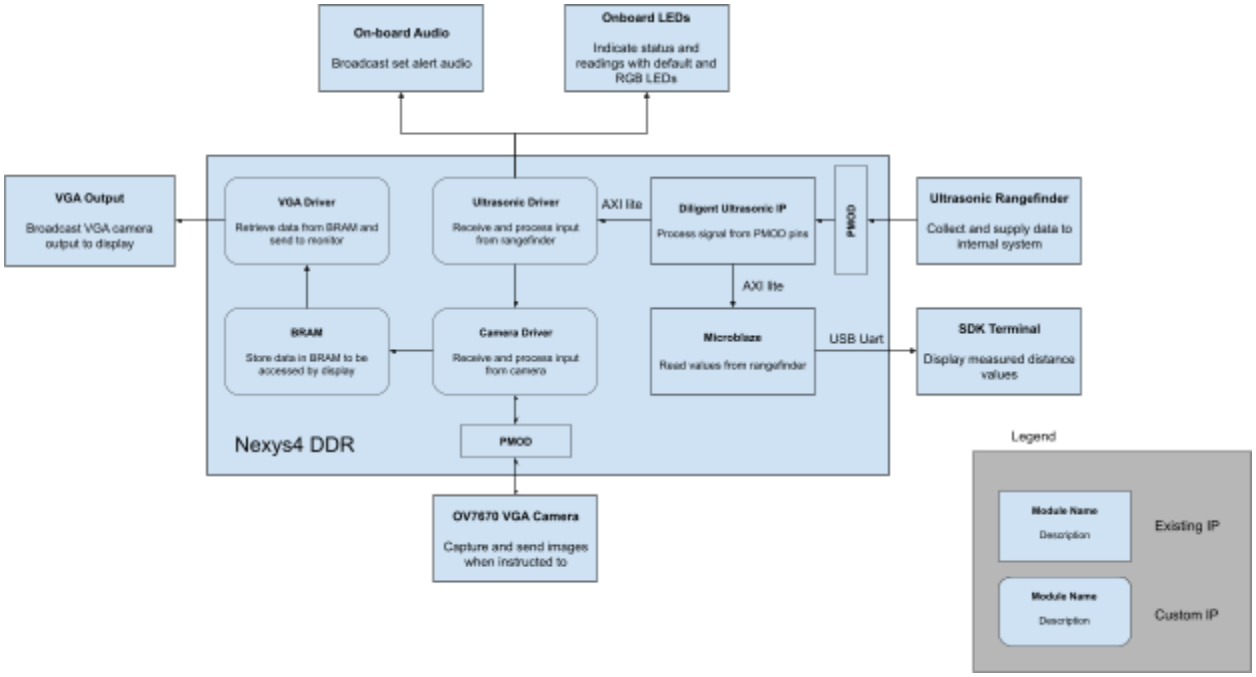
Inspired by these existing projects and the camera-based FPGA designs discussed in the ECE532 course, our team set out to build a multimodal security alert system that combines proximity sensing, event-driven video capture, real-time VGA display, and audio alerting all implemented directly in FPGA hardware for deterministic, low-latency operation.

1.2. Goals

The goal of this project was to design and implement an FPGA-based security alert system on the Nexys 4 DDR board (XC7A100T) with the following functional requirements:

1. Continuous proximity detection using an ultrasonic rangefinder, with configurable distance threshold.
2. Event-driven image capture using an OV7670 camera, triggered when an object enters the detection zone.
3. Real-time VGA display (640×480 @ 60 Hz) of live video with overlay indicators (recording status, camera-off icon, snapshot thumbnail).
4. Audio alerting via the onboard PWM audio output, producing an intermittent 1 kHz alarm tone.
5. Simultaneous activation and deactivation of camera and audio subsystems, coordinated through the block design.

1.3. Block Diagram



1.4. Brief Description of IPs

1.4.1. Xilinx/AMD IPs

Clocking Wizard (clk_wiz_0): Generates 25 MHz pixel clock for VGA and 24 MHz master clock for OV7670 from the 100 MHz board oscillator.

1. MicroBlaze soft processor: Handles ultrasonic sensor data acquisition and outputs real-time distance readings to a serial terminal. Coordinates start/stop signaling to the camera and audio subsystems.
2. AXI Interconnect: Connects MicroBlaze to peripherals including AXI GPIO and sensor IP.

Utility Vector Logic: Inverts the under-threshold signal to power the green LED while nothing is detected.

3. AXI Uartlite: Allows the MicroBlaze to send data externally to the SDK terminal.
4. MicroBlaze Debug Module: Enables debugging of the MicroBlaze processor.

1.4.2. Custom IPs

1. vga_driver_nexys4: VGA timing generation, BRAM read, and multi-layer overlay compositing.
2. Audio alarm (in top.v): Gated 1 kHz square wave generator with open-drain PWM output.
3. camera_read: OV7670 pixel stream capture FSM with start/stop and frame-aligned streaming.

pixel_capture: Frame buffer write controller with snapshot support and deferred reset logic.

4. camera_configure / SCCB_interface / OV7670_config_rom: OV7670 register configuration over SCCB.
5. pixel_capture: This module captures incoming camera pixels, stores a downsampled snapshot first, then continuously writes live video frames to memory while supporting a safe reset at frame boundaries.
6. rec_overlay_rom / off_overlay_rom: Alpha-keyed overlay image ROMs (80×60, 13-bit format).
7. frame_buffer_ram: Parameterized dual-port BRAM (312,000 entries, 12-bit, dual-clock).
sensor_logic_4.0: AXI master interface to read sensor data and determine LED/state output.
Rising edge detector (in top.v): Convert input signal to pulse aligned with rising edge.

1.4.3. Third-Party IPs

1. PmodMAXSONAR_v1_0: Poll ultrasonic sensor and store data in AXI slave modules [1].

1.5. System Changes from Proposal

As the project progressed, several adjustments were made relative to the original proposal:
1. SD card storage removed: The original proposal included SD card storage for captured images. Due to the significant debugging time spent on the camera-to-VGA integration (caused by a defective camera module), the SD card feature was removed to focus on delivering a fully functional core system.
2. OLED display removed: An OLED screen was listed as an add-on option in the proposal for displaying system status. This was not implemented as overlay visuals and distance information printed in the SDK terminal were used instead to provide visual effects and display system data.
3. Python GUI removed: Performance monitoring via a Python GUI was listed as an optional feature. This was not implemented.
4. Pixel format changed from RGB565 to RGB444: During integration, the camera output format was migrated from RGB565 to RGB444 to reduce bandwidth and simplify the downstream pixel processing pipeline.
5. Overlay ROMs downsized from 160×120 to 80×60: The original overlay resolution consumed too many BRAM tiles on the XC7A100T, causing place-and-route failures. Reducing to 80×60 saved approximately 5 RAMB36 blocks.
6. Rangefinder rebuilt without MicroBlaze dependency: The rangefinder was initially integrated using a MicroBlaze-based AXI design. It was later rebuilt to support direct AXI reads for improved modularity, though MicroBlaze was retained for sensor data display and system coordination.
7. Live video streaming added: The original proposal described single-frame capture. The final system supports continuous live video streaming with start/stop control, which was a significant scope expansion.

1.6. Project Complexity

Components	Complexity	Number of Units Needed
Ultrasonic Rangefinder PMOD	0.50	1
OV7670 VGA Camera	1.00	1
Custom IP Block: Sensor Logic	0.50	-
Custom IP Block: Camera Data Processing	0.50	camera_read.v and pixel_capture.v
Custom IP Block: Pulse Generator	0.25	-
VGA Output (without MicroBlaze)	1.00	-
On-board Audio Output Port	0.50	-
Tri-colour LEDS	0.20	-
Total Complexity Score	**4.45**	-

2. Outcome

2.1. Results

Original goals and features in the proposal are:
"The goal of the project is to create a system that detects and reports nearby motion and activity. The design will monitor a specific area for motion, and on detection display a visual warning accompanied by an alert sound, then take a picture of the area to be saved and displayed on a connected monitor. The functional requirements of the design are as follows:

1. Continuous detection of motion within the designated area.
2. Image is captured, saved, and transmitted when a disturbance is detected.
3. Some level of feedback is provided by the system when a detection is made to indicate its presence to anyone nearby.
4. Optionally, the system will provide feedback in both video and audio form.

5. Optionally, performance monitoring of the system will be available with an external system."

Overall, the final system achieved most of the originally proposed features, with some adjustments made during development. The system successfully performs continuous proximity detection, triggers image capture, displays live video with overlay indicators, and produces an audio alert when motion is detected. In addition, the final design expanded beyond the original proposal by supporting live video streaming instead of only single-frame capture. Again, optional features such as SD card storage, OLED display, and Python GUI monitoring were removed due to time constraints and redundant features. The project works reliably and meets its main objectives. If starting over, more time would be allocated for early hardware testing and better resource planning to avoid later design changes.

2.2. Future Work and Improvements

For future improvements, several enhancements can be considered to increase the usability and flexibility of the system. First, the system can be extended to store both static images and live-stream video directly onto an SD card for future reference. This would allow users to review past detections and analyze system performance more conveniently. In addition, implementing wireless communication would improve system accessibility and reduce the need for physical connections. Another potential improvement is to make the detection threshold adjustable through the SDK. This would eliminate the need to regenerate the bitstream each time a parameter change is required, saving both development time and effort. Additionally, adjusting the handling of the LED and start/stop signals from the sensor logic module to improve the stability and reduce flickering between states could boost the reliability of the system. Finally, the measured distance could be displayed on a monitor or an OLED display instead of only showing it in the terminal. This improvement would provide a more user-friendly interface and make the system easier to use in real-world applications.

3. Project Schedule

The table below compares our original milestone plan from the proposal with our actual weekly progress as documented in our milestone reports.

3.1. Milestone Comparison Table

Week	Planned Milestone	Actual Progress	Variance Notes
1	Finalize proposal and project planning. Create and test HDL logic for rangefinder.	Proposal completed. Rangefinder logic written and simulated. VGA driver designed with timing controller. Camera PMOD connections identified.	On track. All members completed individual component design as planned.

		Testbench created for VGA.	
2	Demonstrate stable rangefinder readings on board. Implement threshold detection.	Rangefinder integrated with MicroBlaze and AXI. VGA colour bars and 64×64 image displayed in simulation and on an external monitor. Camera connected to FPGA with ILA debugging.	On track. Rangefinder, VGA, and camera subsystems progressing in parallel.
3	Complete camera bring-up. Verify capture path with valid image data.	Camera SCCB configuration verified. VGA colour bars and 64×64 image confirmed on hardware. Audio alarm (1 kHz, 0.5s ON/OFF) built and verified. Rangefinder rebuilt without MicroBlaze for modularity. Dual-port frame buffer designed.	Slightly ahead on VGA/audio. Camera capture not yet producing frames on display.
4	Complete display bring-up. Show stable VGA output with camera-derived content.	VGA display pipeline verified with BRAM test patterns. Camera-to-BRAM integration attempted but the display remained blank. Sensor + audio integrated and working. ILA used for debugging. Rangefinder LED and audio output implemented.	Behind on camera-to-VGA integration. Blank display issues consumed significant debugging time. BRAM self-test confirmed the VGA path was correct.
5	Complete SD card storage bring-up and verification.	Replaced defective camera module. Fixed camera_read FSM (vsync edge detection). Migrated pixel format from RGB565 to RGB444. Live camera image successfully	SD card storage was removed. Camera integration breakthrough achieved after hardware fault was identified.

		displayed on VGA for the first time.	
6	Integrate all subsystems with correct logic.	Full system integration completed: sensor triggers camera + audio simultaneously. Live 640×480 video with 80×60 snapshot overlay, recording indicator, and camera-off icon. Colour channel mapping corrected. Block design rebuilt in a unified project.	Integration achieved. Overlay ROMs downsized from 160×120 to 80×60 to resolve BRAM overflow. SD card and OLED removed.

3.2. Schedule Analysis

Our project schedule largely followed the original plan for the first three weeks, with the sensor, VGA, and camera subsystems progressing in parallel as intended. The most significant deviation occurred during weeks 4 and 5. The original plan called for stable camera-derived VGA output by week 4 and SD card storage by week 5. In practice, the camera-to-VGA integration was blocked by a defective OV7670 camera module, which was not identified until week 5 after extensive logic-level debugging confirmed that all HDL code was functionally correct. This hardware fault cost approximately one and a half weeks of debugging time.

As a result of the delay, the SD card storage feature was removed from the final system. The OLED display and Python GUI performance monitoring features, which were listed as add-on options in the proposal, were also not implemented. Instead, the team focused on delivering a polished and fully functional core system with live video streaming, overlay compositing, and audio alerting. By week 6, full system integration was achieved and all core features were operational.

After the six-week milestone and before the final demo, additional visual effects were implemented based on the integrated design. More elements were also added to support the final system overview. For example, the actual distance was printed in the SDK terminal, allowing users to observe the distance of objects within the detection zone more clearly.

In retrospect, the schedule would have benefited from an earlier hardware validation step for the camera module. A simple loopback or pixel-counting test at the camera output could have identified the defective unit before the integration phase, potentially saving over a week of debugging effort. Overall, the project timeline and task distribution were well planned. Unnecessary elements were removed when needed, ensuring that the project was completed on time in a unified manner while still meeting the original objectives.

4. Description of the Blocks

This section provides a detailed description of each IP block used in the system. For each block, we specify the module name, source (custom / Xilinx IP / third-party), functionality, modifications made, and testing procedures.

4.1. Ultrasonic Rangefinder Subsystem

4.1.1. Module: PmodMAXSONAR_v1_0 | Source: Third-party (Digilent) [1]

This module drives the ultrasonic sensor through the PMOD port, and collects the raw signal output. This data is stored in an AXI-lite slave register.

4.1.2. Module: sensor_logic_4.0 | Source: Custom (written from scratch)

This module periodically reads from the slave register that stores the raw sensor data. It then converts the data to distance measurements from cycle count using shift-and-add logic instead of division and compares the observed value to the threshold. If above the threshold, a bitwise fill level is created to represent proportionally how close it is, with the upper bound being set to twice the threshold. If below the threshold, the fill level is maxed and an output signal is sent to the rest of the system. Testing was done in stages, first by simulating input values from the sensor and checking if the output signal activated correctly. Then testing was done on the board, with special LED logic set to display a binary representation of the measured distance to verify its accuracy. Once confirmed, the proper LED logic was implemented along with the red and green LEDs, and testing was done on the board again, confirming that the state was properly indicated by the lights.

4.1.3. Module: Utility Vector Logic | Source: Xilinx IP

This module is used as an inverter to flip the sensor logic output signal. The base signal is set to drive the red LED, so it is turned on when an object crosses within the threshold. The inverted signal drives the green LED so it is on in the reverse case. These two signals are also fed to the top level integration files so they can serve as input signals for the camera subsystem. This module was tested in parallel with the sensor logic, as it was a requirement for correct behaviour of the red and green LEDs.

4.1.4. Module: MicroBlaze | Source: Xilinx IP

The MicroBlaze performs a similar task as the sensor logic. It periodically reads from the data register and processes that data into distance measurements. These measurements are then sent externally to SDK to be displayed on the terminal. This is done using third party C code written for the Pmod MAXSONAR IP. This module was tested by comparing to the behaviour of the various output signals from the sensor logic module. First, it was verified that output could be seen on the SDK terminal. Next, multiple threshold values were tested, confirming that the

terminal values matched the expected threshold at the exact time that the system transitioned between states.

4.2. OV7670 Camera Subsystem

Modules: camera_configure.v, SCCB_interface.v, and OV7670_config_rom.v |
Source: The camera configuration modules were developed with reference to [2] OV7670-Verilog repository. For particular differences, start-controlled configuration behaviour was added for integration with the top-level system, and its RGB565 config was replaced with our expected RGB444.

The OV7670 camera subsystem configures the camera and captures a continuous pixel stream for storage in the frame buffer. It operates using a 24 MHz xclk [3] generated by the Clocking Wizard IP and the camera-provided pclk for pixel sampling.

SCCB_interface.v implements the Serial Camera Control Bus (SCCB) communication protocol used to write configuration data to the OV7670 over the sioc and siod lines. This module generates the control timing required for register transactions and provides the low-level interface between the FPGA and the camera's internal control registers. Since reliable SCCB communication depends on proper line behaviour, pull-up resistors were included on the SCCB lines as part of the hardware setup.

OV7670_config_rom.v stores the register initialization sequence used to configure the camera at startup. This sequence defines the operational mode of the sensor, including settings relevant to output formatting and other camera functions required for the project. In this design, the configuration contents were adjusted from the reference implementation so that the camera settings matched the system's expected RGB444-based processing path.

camera_configure.v controls the overall configuration process by stepping through the register values stored in OV7670_config_rom.v and issuing write commands through SCCB_interface.v. It acts as the coordination module for camera initialization. Compared with the referenced design, this module was modified to support start-controlled configuration behaviour, allowing the camera setup process to be triggered in a controlled way from the top-level system rather than running automatically at power-up.

Testing of the camera configuration subsystem was performed using both on-board LEDs and the Integrated Logic Analyzer (ILA). LEDs were used for quick observation of major status signals, while the ILA provided more detailed verification of signals such as pclk, vsync, href, and SCCB-related activity during debugging. These tests were important in confirming correct camera initialization and in diagnosing hardware communication issues.

4.3. Camera Data Processing

4.3.1. Module: camera_read.v | Source: Custom (written from scratch)

This module controls the acquisition of raw data from the camera sensor and converts it into valid pixel data. It uses a finite state machine to manage different stages, including idle, frame alignment, and pixel capture. The module waits for a start signal before beginning the streaming process and ensures that capturing begins at a clean frame boundary. It also detects vertical synchronization signals to identify frame start and end conditions. During pixel capture, the module combines two 8-bit data inputs into one 16-bit pixel value and asserts a valid signal when a full pixel is ready. Additionally, the module supports a stop signal that allows the system to safely stop streaming after the current frame is completed. This design improves reliability and ensures smooth image acquisition.

4.3.2. Module: pixel_capture.v | Source: Custom (written from scratch)

This module is responsible for receiving pixel data from the camera and storing it into the frame buffer memory. It processes incoming pixel data using the pixel clock and tracks pixel positions using horizontal and vertical counters. During the initial stage, the module captures a lower-resolution snapshot by sampling one pixel out of every eight pixels in both horizontal and vertical directions. This reduces memory usage and allows a quick preview image to be created. After the snapshot is completed, the module switches to live streaming mode, where every pixel is written sequentially into memory. In addition, the module handles reset conditions and ensures that memory writing only occurs when valid pixel data is available. This helps maintain stable operation and prevents corrupted frames.

4.4. Block Design and System Integration

Module: Rising edge detector | Source: Custom (written from scratch)

This simple module takes a switch-like signal output from the ultrasonic subsystem and converts it to a single cycle pulse, to serve as the input for the audio and camera subsystems. It also does this for the inverted output to serve as reset signals for those systems as well. This is done by using the AND operator on the signal and an inverted, one-cycle delayed version of itself. For stability purposes, a three-cycle buffer was added to confirm that the pulse is only sent when the signal remains active for multiple cycles. This module was tested by replacing the button input and debouncer logic for the working camera subsystem with this new input signal and verifying that it still worked.

4.5. VGA Display Subsystem

Module: vga_driver_nexys4.v | Source: Custom (written from scratch)

The VGA driver generates standard 640×480 @ 60 Hz timing signals (hsync, vsync, blanking) and reads 12-bit RGB444 pixel data from a dual-port BRAM frame buffer. It operates on a 25 MHz pixel clock produced by the Clocking Wizard IP. BRAM addresses are computed using

shift-and-add logic to avoid multiplier usage: $y \times 640 + x$ is computed as $(y << 9) + (y << 7) + x$, and $y \times 80 + x$ as $(y << 6) + (y << 4) + x$.
The driver implements a multi-layer overlay system with priority-based pixel multiplexing:
Snapshot overlay (top-left, 80×60): Displays the first captured frame from BRAM address 307,200. A 1-pixel white border separates it from the live video.
Recording indicator (bottom-right, 80×60): Alpha-keyed icon from rec_overlay_rom displayed at (560, 420) when streaming is active. Bit [12] of the 13-bit ROM output serves as the alpha flag.
Camera-off icon (center, 80×60): Icon from off_overlay_rom displayed against a dark grey background (0x111) when the camera is idle. Originally designed at 160×120 but reduced to 80×60 to save approximately 5 RAMB36 blocks.
All reads use a 1-cycle pre-fetch pipeline: addresses are issued one clock ahead and region flags are delayed by one register stage to align with BRAM/ROM registered outputs.
Testing: VGA timing was verified in Vivado simulation via testbench. On-board validation was performed using the BRAM self-test module (top_vga_colorbars.v), which writes known 12-bit test patterns into port A while the VGA driver reads from port B.

4.6 Audio Alarm Subsystem

Module: Audio alarm logic in top.v (originally developed as audio_alarm_pattern.v) | Source: Custom (written from scratch)
The audio alarm generates an intermittent 1 kHz square wave with a 0.5-second ON / 0.5-second OFF gating pattern. A 26-bit counter divides the 100 MHz system clock to produce the 0.5 s gate interval (50,000,000 cycles), and a 16-bit counter generates the 1 kHz tone by toggling every 50,000 cycles. When either the alarm enable signal is low or the gate is in its OFF phase, the tone counter is held at zero to prevent residual clicks.
The audio output uses open-drain signaling on the AUD_PWM pin (drives low or high-Z, never high) as required by the Nexys 4 DDR audio hardware. The amplifier is enabled via AUD_SD whenever the alarm is active. In the final system, the alarm is controlled by the same bd_start_pulse / bd_stop_pulse signals that control the camera, ensuring simultaneous activation.
Testing: Initially developed and verified as a standalone bitstream (nexys4_audio_alarm_top.v) with SW0 as the enable input and a dedicated .xdc constraint file. After verification, refactored into a reusable module with a generic alarm_en input for integration.

4.7 Frame Buffer

Module: frame_buffer_ram.v | Source: Custom (written from scratch)
A parameterized dual-port BRAM module with independent write and read clocks. Configured with 19-bit addresses, 12-bit data width, and a depth of 312,000 entries (307,200 for the 640×480 live frame + 4,800 for the 80×60 snapshot). The write port operates on the camera pixel clock (pclk) and the read port on the 25 MHz VGA clock (clk_25), enabling simultaneous camera capture and VGA display without clock domain conflicts.

4.8 Overlay ROMs

Modules: rec_overlay_rom.v, off_overlay_rom.v | Source: Custom (written from scratch)
Each ROM stores 4,800 entries (80×60 pixels) in 13-bit format: bit [12] is the alpha flag (1 = opaque, 0 = transparent), bits [11:8] are blue, [7:4] green, and [3:0] red. ROM contents are loaded from .mem files generated from PNG images. Both ROMs use the Vivado block ROM synthesis style attribute.

5. Description of Design Tree

Repository at: https://github.com/noahmarshall415/ECE532-Project-Repo
This repository includes all the necessary source, constraint, block diagram, and memory files used in the project. Also included is an IP repo folder that contains imported IPs, both custom and third-party, that are used as well. The project_1.tcl file can be run in the Vivado terminal to build the project in its entirety. This can be done by navigating to the source directory in the terminal and using the command: "source project_1.tcl". Additionally, the final group report for the project is located in the doc folder and a few unused or temporary files mentioned in the report are included as well.

6. Tips and Tricks

1. Use a BRAM self-test module for VGA debugging. When integrating a camera with a VGA display through a shared frame buffer, a blank screen can be caused by any stage in the pipeline. Writing a standalone module that fills the BRAM with a known test pattern (bypassing the camera entirely) and reading it back through the VGA driver is an extremely effective way to isolate the fault. Our top_vga_colorbars.v module demonstrates this approach.
2. Do not overlook hardware faults. After extensive debugging of our HDL code revealed no logical errors, we discovered that the OV7670 camera module itself was physically defective. Replacing it immediately produced a correct image. If your simulation is correct and your BRAM read path works with test data, consider swapping the hardware component before investing more time in logic debugging.
3. Validate camera output independently before integration. Before connecting the camera to the full VGA pipeline, verify that the camera is outputting valid pixel data by using ILA to probe key signals (pixel_valid, frame_done, vsync) at the camera output.
4. Plan BRAM usage early. BRAM is a limited resource on the XC7A100T (135 RAMB36 blocks). Our overlay ROMs were originally designed at 160×120 pixels, which caused place-and-route failures. Estimating total BRAM consumption before implementation can prevent late-stage surprises.

7. Video

This is the link to the demonstration of the designed security system, which functions as a security camera and can be installed in locations such as a front door:
https://drive.google.com/file/d/19CwpFvqmgcNlPqxE1jszhJBNMtFm3J-n/view?usp=sharing.

8. References

[1] Digilent, “vivado-library: Releases,” GitHub, 2021. Available: https://github.com/Digilent/vivado-library/releases

[2] Wbraun, “OV7670-Verilog,” GitHub. Available: https://github.com/westonb/OV7670-Verilog

[3] OmniVision Technologies, “OV7670/OV7171 CMOS VGA (640×480) CameraChip™ Sensor Datasheet,” Aug. 2006. Available: https://web.mit.edu/6.111/www/f2016/tools/OV7670_2006.pdf